The Life of a Process

Just as organisms have a life cycle, so do processes. They are born, they carry out assigned tasks, they go to sleep and, finally, die or are killed. In the course of its life cycle, a process goes through various stages. This article outlines the life of a process in Linux.

A process is the running entity of a program, which has various information like the process's address space, pending signals, open files, etc. So there must be one structure to store this information. Linux provides a structure called *task_struct*, which is used to store process information. This is called the process descriptor. This structure is defined in the *<linux/sched.h>* header file. This is a large data structure that has a size of 1.7KB.

Note: Linux stores all *task_structs* in a circular doubly-linked list and not in a static array. This is called the task list.

Birth of a process

The life of a process is created by another process, which means a process must have its parent process. A fork API is used to create a process from the user space. The API gives a clone system call, which calls the *do_fork* function, which in turn calls the *copy_process* function, inside which is the *dup_task_struct* that creates the new process. Creating a new process means creating a new kernel stack, new *task_struct* and new *thread_info*.

Linux does not create a copy of the parent process in order to create a child but follows the copy on write (COW) concept. That means Linux creates a new address space for newer processes only when the current address space goes through the write operation. Before that, they share common resources in a read-only condition. A new process or child has a new PID, created by the *alloc_pid()* function called from the *copy_process()*, and all pointers point to parent structures.

If the purpose of creating a process is not to clone the parent process, then a process will run a different entity of the program that is assigned by exec API.

Note: After successful execution of *copy_process()*, the kernel runs the child process first and calls *exec()* immediately thereafter to avoid a COW overhead because, if the parent starts writing on the address space, the kernel has to create a separate copy for the child.

Before kernel 2.6, *task_struct* was created statically and the kernel stack had a pointer to this structure. Now *task_struct* is created by the slab allocator and the kernel stack